

TRIBUTACOS · SHELLAQUILES.ORG

tribuTACOS

Documentación Técnica

Generado automáticamente · Agosto 2026

Documentación técnica oficial de la arquitectura, modelo de datos, algoritmos fiscales y especificaciones de API de la plataforma tribuTACOS.

Índice General

El sistema se documenta a través de los siguientes módulos técnicos:



Módulos de Documentación

MÓDULO	DOCUMENTO	ALCANCE Y CONTENIDO
01	01. Arquitectura General	Arquitectura desacoplada, pipeline de ingesta de comprobantes y flujo de datos.
02	02. Modelo de Datos Relacional	Diagramas entidad-relación, esquemas relacionales, indexación y almacenamiento de caché.
03	03. Motor Fiscal y Algoritmos	Fundamentos matemáticos y legales: Art. 96, 106, 151 y 152 de la LISR, y Art. 5 y 6 de la LIVA.

04	04. Catálogos SAT y Taxonomía	Clasificación de claves UNSPSC del SAT y algoritmo de resolución en 4 niveles.
05	05. Especificación de API REST	Contratos OpenAPI, esquemas de entrada/salida y códigos de respuesta HTTP.
06	06. Frontend y Componentes	Arquitectura cliente en Next.js 15, sistema de diseño semántico y exportación de datos.
Anexos	Documentación Funcional / Resumen Técnico	Especificaciones funcionales y resumen ejecutivo de despliegue.

Ejecución del Entorno de Desarrollo

Requisitos Previos

- Python 3.11 o superior
- Node.js 18 o superior
- GNU Make

Comandos de Inicialización

```
# Inicialización completa (instalación de dependencias y base de datos con datos de prueba)
make setup

# Ejecución de servidores de desarrollo (Backend :8010 y Frontend :3000)
make dev

# Ejecución de suite de pruebas automatizadas (backend + control_panel)
make test
```

Servicios Standalone

```
# Capturas del Panel de Operaciones (Tkinter)
make screenshots-gui
```

```
# Backend FastAPI (Puerto 8010)
cd backend && source venv/bin/activate && uvicorn app.main:app --host 0.0.0.0 --port 8010 -
-reload

# Frontend Next.js (Puerto 3000)
cd frontend && npm run dev
```

Aviso Legal / Disclaimer

tribuTACOS es una plataforma de análisis, proyección y simulación fiscal basada en la interpretación algorítmica de comprobantes digitales (CFDI) y la legislación mexicana (LISR y LIVA). Los cálculos, proyecciones y determinaciones presentados son de carácter estrictamente estimativo e informativo, no constituyen asesoría fiscal, contable o legal vinculante, y no sustituyen las determinaciones o declaraciones oficiales del Servicio de Administración Tributaria (SAT).

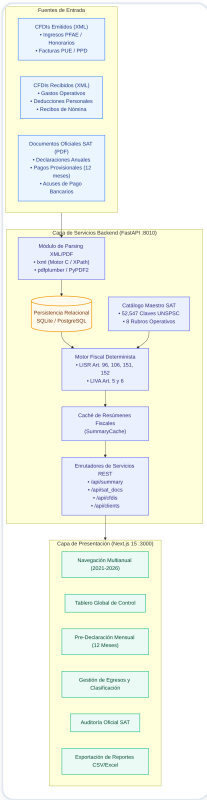
tribuTACOS — 01. Arquitectura General y Ecosistema Técnico

Versión de Referencia del Sistema: Esta documentación técnica describe la arquitectura y especificación de **tribuTACOS v1.1.0-rc.4 RC**.

Especificación formal de la arquitectura del sistema, componentes de software, pipeline de procesamiento de datos y estructura de directorios.

1. Visión General del Sistema

tribuTACOS es una plataforma web de inteligencia fiscal y auditoría automatizada diseñada para procesar Comprobantes Fiscales Digitales por Internet (CFDI 3.3 y 4.0 en XML) y declaraciones oficiales del Servicio de Administración Tributaria (en PDF). El sistema integra un motor fiscal determinista que calcula pre-declaraciones provisionales mensuales y la determinación anual del Impuesto Sobre la Renta (ISR) e Impuesto al Valor Agregado (IVA), en apego a la Ley del Impuesto Sobre la Renta (LISR), Ley del Impuesto al Valor Agregado (LIVA) y el Código Fiscal de la Federación (CFF).



2. Stack Tecnológico

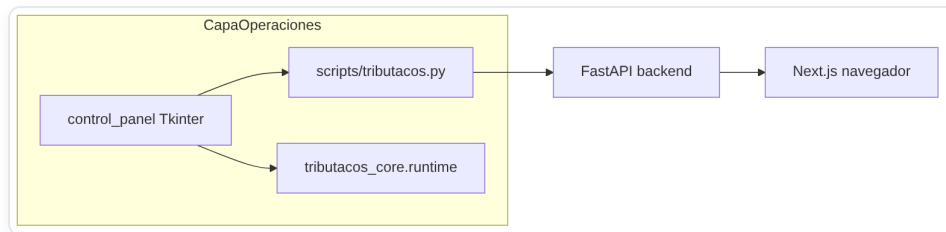
CAPA	COMPONENTE	JUSTIFICACIÓN TÉCNICA
Backend Core	Python 3.11+	Lenguaje base para ejecución de lógica contable, aritmética fiscal y transformaciones de datos.

Framework Web	FastAPI + Uvicorn	Framework asíncrono basado en ASGI, tipado estricto con Pydantic v2 y generación de OpenAPI.
Persistencia	SQLAlchemy 2.0 + SQLite / PostgreSQL	Capa de abstracción de datos relacional con soporte transaccional ACID e indexación optimizada.
Motor de Parsing XML	<code>lxml</code>	Parser basado en libxml2 con optimización C para procesamiento masivo de XMLs fiscales.
Motor de Parsing PDF	<code>pdfplumber</code> + <code>PyPDF2</code>	Extracción estructurada de tablas, metadatos y valores fiscales de documentos oficiales SAT.
Frontend	Next.js 15 + React 19	Arquitectura App Router para renderizado eficiente de interfaces analíticas complejas.
Diseño y Estilos	Tailwind CSS	Sistema de utilidades CSS con tipografía formal, alto contraste y paletas semánticas estructuradas.
Exportación de Datos	Vanilla JavaScript (<code>csvExport.js</code>)	Generación en cliente de archivos CSV con codificación UTF-8 BOM para compatibilidad total con hojas de cálculo.
Generador de PDFs	Pandocquiles by sheillaquiles.org	Pipeline de compilación Markdown a PDF con renderizado de diagramas Mermaid, incrustación de recursos Base64 y tematización CSS editorial.
Panel de Operaciones	Tkinter (Python)	Orquestación de arranque standalone, carpetas de ingesta, PDFs SAT locales, respaldos y utilidades fuera de la interfaz web fiscal.
Runtime compartido	<code>tributacos_core</code> + <code>scripts/tributacos.py</code>	Rutas, modo de distribución, diagnóstico y comandos compartidos entre CLI, panel GUI y empaquetado PyInstaller.

2.1 Capa de operaciones y runner compartido

La **interfaz fiscal** (dashboard, ingesta XML en modal, CSV, conciliación SAT) vive en el navegador (Next.js). El **Panel de Operaciones** (`control_panel/` , Tkinter) no replica esa UI: orquesta arranque

del servidor standalone, apertura de carpetas de ingesta, procesamiento local de XML/PDF, respaldos y utilidades de sistema.



- `scripts/tributacos.py` : fuente única de comandos; `make X` y el panel invocan el mismo runner.
- `tributacos_core/runtime.py` : rutas del proyecto, modo (`installed` / `docker` / `dev`), URL de la app y ficha técnica.
- `control_panel/` : capas `domain/` (ventana y servidor local), `config/` (`copy` , `constants` , `catalog` , `theme`), `ui/views/` (builders por pestaña) e `infra/bootstrap` (paths de importación).

Convenciones de desarrollo del panel: ver `control_panel/README.md` .

3. Pipeline de Procesamiento de Datos

3.1 Ingesta y Validación de Estructura

- 01 Los archivos XML son recibidos y validados contra los esquemas estándar del SAT (CFDI versión 3.3 o 4.0).
- 02 El parser extrae los atributos esenciales:
 - **UUID**: Folio Fiscal único del Comprobante Fiscal Digital por Internet.
 - **Tipo de Comprobante**: `I` (Ingreso), `E` (Egreso), `N` (Nómina), `P` (Pago).
 - **Condiciones de Pago**: `PUE` (Pago en una sola exhibición) o `PPD` (Pago en parcialidades o diferido).
 - **Partidas y Conceptos**: Clave de Producto o Servicio, desglose de base imponible, traslados de IVA (16%, 8%, 0%, Exento) y retenciones (ISR 10%, 1.25%, IVA 10.6667%).
 - **Complemento de Nómina (versión 1.2)**: Días laborados, percepciones gravadas y exentas (Art. 93 LISR), deducciones y retención de ISR (Art. 96 LISR).

3.2 Normalización y Persistencia Relacional

- Cada comprobante es registrado en la tabla `cfdis` asignado a un `client_id` .
- Se ejecuta un control de integridad para prevenir duplicados mediante restricciones de unicidad por `UUID` .
- Los metadatos extendidos del CFDI se almacenan en formato JSON normalizado dentro del campo `parsed_data` .

3.3 Ejecución del Motor Fiscal

Al consultar el resumen fiscal anual (`/api/summary?year=YYYY`), el motor procesa:

- 01 Masa salarial anual, desglose de percepciones gravadas/exentas y retenciones de nómina.
- 02 Ingresos mensuales por actividades profesionales y empresariales (PFAE).
- 03 Clasificación de partidas de gasto en los 8 rubros operativos estandarizados.

- 04 Determinación de deducciones personales y aplicación de los topes legales del Art. 151 LISR.
 - 05 Cálculo de pagos provisionales mensuales de ISR (Art. 106 LISR) y determinación de IVA mensual con arrastre de saldos a favor (Art. 5 y 6 LIVA).
 - 06 Determinación anual de ISR mediante la tarifa progresiva del Art. 152 LISR.
-

4. Estructura del Repositorio

```
tributacos/
├── backend/                                # Servicios de Backend y Lógica de Negocio
│   ├── app/
│   │   ├── catalogos/                    # Catálogos SAT y Clasificación Taxonómica
│   │   │   ├── taxonomia.py             # Definición de rubros y reglas de clasificación
│   │   │   ├── sat_catalogo.py          # Algoritmo de resolución jerárquica en 4 niveles
│   │   │   ├── seed.py                  # Poblado del catálogo maestro UNSPSC (52,547 claves)
│   │   │   └── seed_fiscal.py           # Poblado de tarifas del Art. 152 LISR y factores UMA
│   │   ├── cfdis/                       # Módulos del Motor Fiscal y Procesamiento CFDI
│   │   │   ├── calculators/             # Calculadoras fiscales de dominio puro
│   │   │   │   ├── tarifas.py           # Tarifa del Art. 152 LISR y desglose marginal
│   │   │   │   ├── nomina.py            # Sueldos, retenciones e ingresos exentos
│   │   │   │   ├── honorarios.py        # Ingresos PFAE y analítica de clientes
│   │   │   │   ├── gastos.py            # Egresos deducibles y matriz mensual
│   │   │   │   ├── deducciones.py       # Deducciones personales y topes legales
│   │   │   │   ├── intereses.py         # Intereses reales del sistema financiero
│   │   │   │   └── simulador_sat.py     # Determinación anual y pre-declaración mensual
│   │   ├── engine.py                   # Orquestador del resumen fiscal consolidado
│   │   ├── parser.py                   # Parser de comprobantes CFDI 3.3/4.0
│   │   ├── storage.py                  # Gestión de almacenamiento e invalidación de caché
│   │   ├── schemas.py                  # Esquemas Pydantic v2 de validación
│   │   └── router.py                   # Endpoints REST de CFDIs y resúmenes
│   ├── sat_docs/                       # Módulo de Documentos Oficiales SAT (PDF)
│   │   ├── parser.py                   # Extractor de declaraciones y acuses SAT
│   │   ├── importer.py                 # Ingesta y persistencia de documentos SAT
│   │   └── router.py                   # Endpoints de consulta y conciliación SAT
│   ├── seeds/                          # Módulos de Generación y Poblado de Datos
│   │   ├── seed_demo.py                # Importador y exportador de fixtures
│   │   ├── recalculate_dataset.py       # Generador determinista del dataset fiscal
│   │   └── demo_dataset.json.gz         # Fixture comprimido de prueba
│   ├── auth/                           # Módulo de autenticación
│   ├── database.py                     # Conexión SQLAlchemy y gestión de sesiones
│   ├── models.py                       # Definición de modelos ORM
│   ├── cli.py                          # Interfaz de línea de comandos unificada
│   ├── config.py                       # Configuración del entorno
│   └── main.py                         # Inicialización de la aplicación FastAPI
├── tests/                              # Suite de pruebas automatizadas Pytest
├── tributacos.db                       # Base de datos SQLite
├── requirements.txt                     # Dependencias de Python
├── frontend/                           # Aplicación Cliente Next.js 15
│   ├── src/
│   │   ├── app/                        # Enrutador App Router (layout.js, page.js, globals.css)
│   │   ├── components/                 # Componentes de interfaz de usuario
│   │   │   ├── ui/                     # Componentes base reutilizables
│   │   │   ├── nomina/                 # Vistas de nómina y recibos de sueldos
│   │   │   ├── honorarios/             # Vistas de honorarios y facturación PFAE
│   │   │   ├── egresos/                 # Vistas de gastos y desglose de rubros
│   │   │   ├── deducciones/             # Vistas de deducciones personales y cálculo anual
│   │   │   ├── PreDeclaracionMensualSection.jsx # Matriz mensual de 12 meses
│   │   │   └── ConciliacionSatSection.jsx # Comparativa de auditoría SAT
│   │   ├── SatUI.jsx                   # Exportación modular de componentes
│   │   ├── csvExport.js                 # Módulo de exportación CSV con UTF-8 BOM
│   │   ├── App.jsx                     # Componente principal de navegación y estado
│   │   └── index.css                    # Estilos base y tokens de diseño
│   ├── package.json                    # Dependencias de Node.js
│   └── next.config.mjs                  # Configuración de Next.js
├── docs/                               # Documentación Técnica del Sistema
└── img/                                # Capturas del Panel de Operaciones (guía de instalación)
```

```
|   └─ INSTALACION_USUARIO.md
|─ manual_usuario/           # Manual de usuario y capturas web
|─ tributacos_core/         # Runtime compartido (rutas, modo, diagnóstico)
|   └─ runtime.py
|─ control_panel/           # Panel de Operaciones Tkinter
|   └─ domain/              # panel.py, server.py, models.py
|   └─ config/              # copy.py, constants.py, catalog.py, theme.py
|   └─ ui/views/            # builders por pestaña
|   └─ infra/               # bootstrap (sys.path)
|   └─ scripts/             # capture_screenshots.py
|   └─ tests/               # test_config.py, test_gui_smoke.py
|─ scripts/                 # Runner CLI y launchers (.bat, .sh, .pyw)
|   └─ tributacos.py
|   └─ runtime.py           # shim → tributacos_core
|─ packaging/              # PyInstaller, Inno Setup y bundle Docker
|─ VERSION                 # SemVer fuente unica
|─ Makefile                # Fachada de scripts/tributacos.py (make X == python scripts/
tributacos.py X)
└─ levantar_proyecto.sh    # Alias de python3 scripts/tributacos.py dev
```

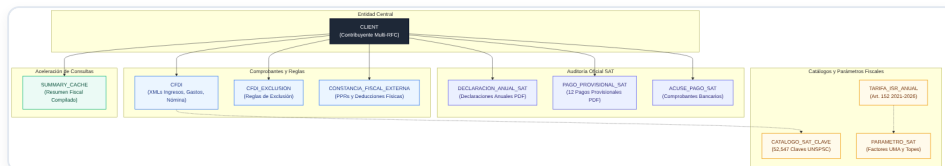
5. Aviso Legal / Disclaimer

tribuTACOS es una plataforma de análisis, proyección y simulación fiscal basada en comprobantes digitales (CFDI) y las leyes tributarias aplicables. Los resultados generados son de naturaleza puramente informativa y proyectiva, no representan asesoría contable o fiscal vinculante, y no reemplazan las declaraciones oficiales presentadas ante el Servicio de Administración Tributaria (SAT).

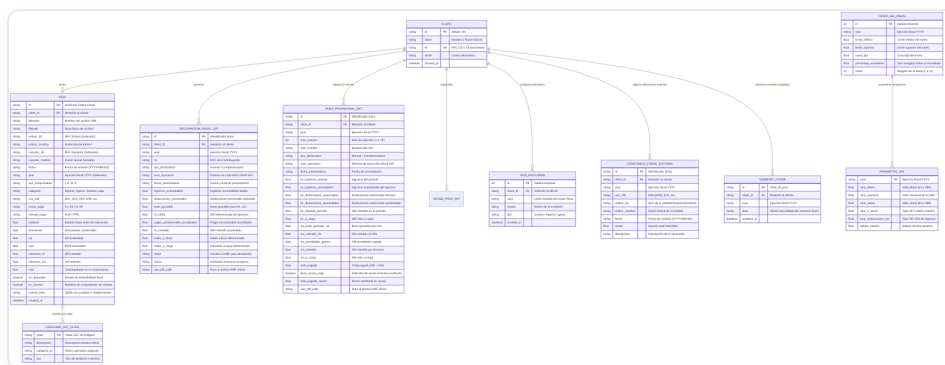
tribuTACOS — 02. Modelo de Datos Relacional y Esquema SQLAlchemy

Diseño de base de datos relacional, diagramas entidad-relación (ERD), modelos SQLAlchemy, índices de rendimiento y diccionario de datos.

1. Esquema Relacional de Entidades



2. Diagrama Entidad-Relación Detallado (ERD)



3. Índices de Rendimiento y Optimización

Para garantizar tiempos de respuesta inferiores a 15 ms en consultas analíticas sobre volúmenes elevados de comprobantes, se definen los siguientes índices en SQLAlchemy:

```
# Declaración en backend/app/models.py
__table_args__ = (
    Index('idx_cfdi_cliente_year', 'cliente_id', 'year'),
    Index('idx_cfdi_emisor_fecha', 'emisor_rfc', 'fecha'),
    Index('idx_cfdi_receptor_fecha', 'receptor_rfc', 'fecha'),
    Index('idx_cfdi_categoria', 'categoria'),
)
```

4. Estructura del Campo Desestructurado (parsed_data)

Cada comprobante fiscal almacena en su columna `parsed_data` un objeto JSON estandarizado con la información analizada del XML:

```
{
  "uuid": "4A1B2C3D-E4F5-6789-ABCD-EF0123456789",
  "emisor_rfc": "ITN150420AA1",
  "emisor_nombre": "INNOVACION TECNOLOGICA DEL NORTE SA DE CV",
  "receptor_rfc": "SHLL250825XYZ",
  "receptor_nombre": "Sheila Shellaquiles Ortega",
  "regimen_fiscal_emisor": "601",
  "regimen_fiscal_receptor": "605",
  "conceptos": [
    {
      "clave": "81111508",
      "desc": "Desarrollo de software y arquitectura backend en Python",
      "imp": 35000.00,
      "subtotal": 35000.00,
      "descuento": 0.00,
      "tasa_iva": 0.16,
      "iva": 5600.00,
      "ret_isr": 3500.00,
      "ret_iva": 3733.33
    }
  ],
  "nomina_detalle": {
    "dias_pagados": 15.0,
    "fecha_inicial_pago": "2024-01-01",
    "fecha_final_pago": "2024-01-15",
    "salario_base_cot_apor": 1350.00,
    "salario_diario_integrado": 1420.50,
    "percepciones": [
      { "tipo": "001", "concepto": "Sueldos y Salarios", "gravado": 15000.00, "exento": 0.0
    }
  ],
  "deducciones": [
    { "tipo": "001", "concepto": "Seguridad Social IMSS", "total": 450.00 },
    { "tipo": "002", "concepto": "Retención de ISR", "total": 2150.00 }
  ]
}
}
```

5. Estrategia de Caché e Invalidación (`summaryCache`)

La tabla `summary_cache` almacena el payload precalculado del resumen fiscal para proporcionar respuestas de baja latencia a la capa de frontend.

Eventos de Invalidación y Reconstrucción:

- 01 Ingesta, modificación o eliminación de un registro en `cfdis` .
- 02 Registro o actualización de documentos oficiales en `declaraciones_anuales_sat` o `pagos_provisionales_sat` .
- 03 Modificación de exclusiones fiscales en `cfdi_exclusions` .
- 04 Ejecución del comando de regeneración de base de datos (`make db-fresh`).

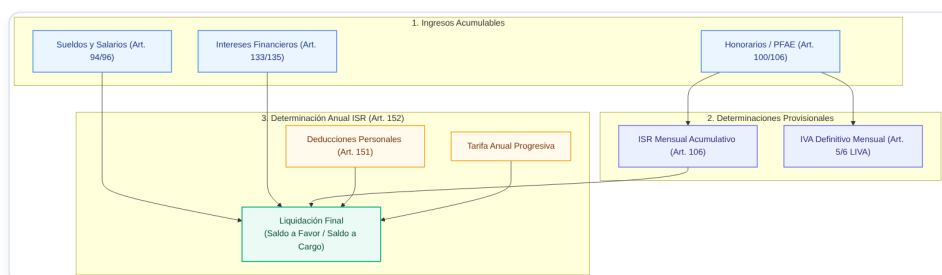
tribuTACOS — 03. Motor Fiscal y Algoritmos LISR/LIVA

Modelación matemática, fundamentos legales y algoritmos deterministas para pagos provisionales de ISR e IVA, nómina y determinación anual.

1. Fundamentos Legales y Módulos de Cálculo

El motor fiscal implementado en `backend/app/cfdis/calculators/` y coordinado por `backend/app/cfdis/engine.py` opera como un conjunto de funciones matemáticas deterministas desacopladas de la persistencia:

- `tarifas.py` : Cálculo de la tarifa del Art. 152 LISR con desglose de límite inferior, cuota fija, porcentaje excedente e impuesto marginal.
- `nomina.py` : Cálculo de sueldos, desglose de percepciones gravadas y exentas (Art. 93 LISR), deducciones y serie temporal de recibos.
- `honorarios.py` : Facturación PFAE emitida, serie de 12 meses, concentración por cliente y mezcla de conceptos por clave SAT.
- `gastos.py` : Deducibilidad operativa, matriz mensual de egresos y asignación de rubros SAT.
- `deducciones.py` : Deducciones personales (Art. 151 LISR), integración de planes de retiro (PPR), gastos médicos y cálculo del doble tope legal.
- `intereses.py` : Intereses nominales, cálculo de interés real y retenciones financieras.
- `simulador_sat.py` : Pre-declaración mensual provisional y determinación anual con desglose paso a paso.



2. Pagos Provisionales de ISR (Art. 106 LISR)

Para personas físicas con Actividad Empresarial y Profesional, el pago provisional de ISR se calcula de forma acumulativa mes a mes dentro del ejercicio fiscal:

2.1 Base Gravable Acumulada:

$$\text{Base Gravable Acumulada}(m) = \text{MAX}(0, \sum \text{Ingresos PFAE}(1..m) - \sum \text{Gastos Deducibles}(1..m))$$

2.2 Tarifa Mensual Acumulada:

Para calcular el impuesto causado acumulado al mes `m`, se anualiza la base y se aplica la tarifa del Art. 152 multiplicada por la proporción del periodo `m / 12`:

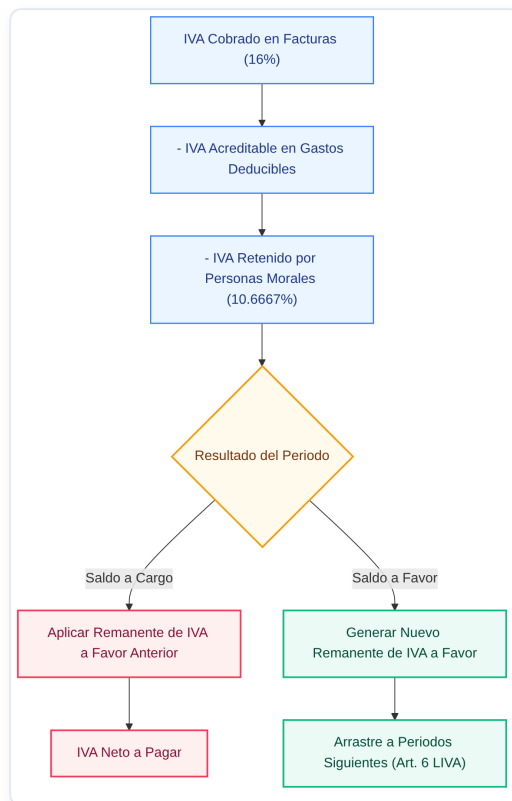
$$\text{ISR Causado Acumulado}(m) = \text{TarifaAnual}(\text{Base}(m) \times 12 / m) \times (m / 12)$$

2.3 ISR Neto a Pagar del Mes:

$$\text{ISR a Pagar}(m) = \text{MAX}(0, \text{ISR Causado Acumulado}(m) - \sum \text{Pagos Prov Anteriores} - \sum \text{ISR Retenido})$$

3. IVA Definitivo y Acreditamiento de Saldos a Favor (Art. 5 y 6 LIVA)

A diferencia del ISR, el IVA es un impuesto definitivo de causación mensual que permite el acreditamiento de remanentes a favor generados en periodos anteriores:



4. Topes de Deducciones Personales (Art. 151 LISR)

Las deducciones personales (honorarios médicos [D01](#), gastos dentales [D02](#), gastos hospitalarios [D03](#), intereses reales hipotecarios [D05](#), primas de seguros [D07](#), colegiaturas) están sujetas a un límite general:

$$\text{Tope Legal General} = \text{MIN}(15\% \times \text{Total Ingresos Anuales}, 5 \times \text{UMA Anual})$$

4.1 Excepciones y Reglas Específicas:

- **Planes Personales de Retiro (PPR - Fracc. V):** Cuentan con un límite independiente de hasta el **10% de los ingresos acumulables o 5 UMAs anuales**, adicional al tope general.
- **Seguro de Gastos Médicos Mayores (SGMM - Fracc. VI):** Se computa dentro del límite general sin subtopes específicos.

4.2 Valores de 5 UMAs Anuales por Ejercicio:

- **2022:** \$178,938.00 MXN
- **2023:** \$189,222.00 MXN
- **2024:** \$198,031.80 MXN

- **2025:** \$209,113.65 MXN
- **2026:** \$220,614.90 MXN

5. Tarifa Progresiva Anual (Art. 152 LISR)

La determinación anual aplica la tarifa progresiva sobre la base gravable:

$$\text{Base Gravable Anual} = \text{Ingresos Acumulables Totales} - \text{Deducciones Personales Aceptadas}$$

$$\text{ISR Causado Anual} = \text{Cuota Fija} + (\text{Base Gravable} - \text{Límite Inferior}) \times \text{Tasa Marginal}$$

$$\text{Resultado Final} = \text{ISR Causado Anual} - \text{Retenciones Nómina} - \text{Retenciones PFAE} - \text{Pagos Provisionales ISR}$$

- Si $\text{Resultado Final} < 0$: **Saldo a Favor** (sujeto a devolución o compensación).
- Si $\text{Resultado Final} > 0$: **Saldo a Cargo** (impuesto a liquidar ante el SAT).

tribuTACOS — 04. Catálogos SAT y Taxonomía de Rubros Operativos

Estructura de clasificación basada en el catálogo UNSPSC del SAT, esquema de agregación en 8 rubros operativos y algoritmo de resolución jerárquica en 4 niveles.

1. Clasificación en 8 Rubros Operativos

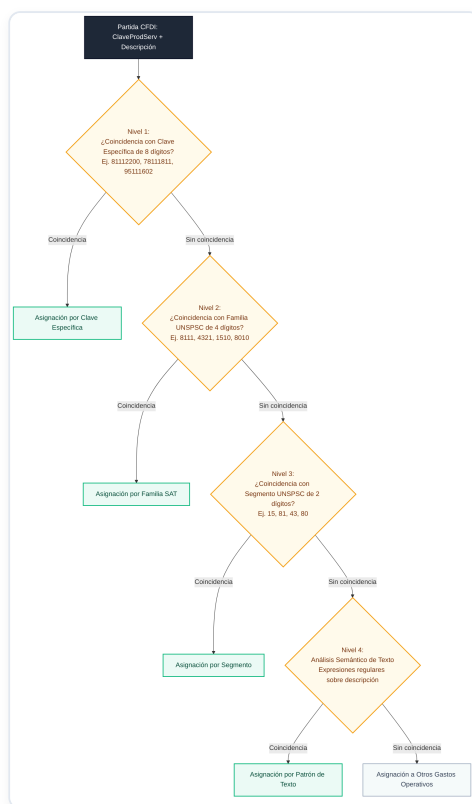
El catálogo oficial de productos y servicios del SAT (`c_ClaveProdServ`) contiene más de 52,000 registros que describen bienes y servicios individuales. Con el fin de estructurar la información contable y facilitar el análisis de deducibilidad para personas físicas con actividad empresarial y profesional, el sistema agrega todas las partidas de gasto en 8 rubros operativos estandarizados:

ID	RUBRO OPERATIVO	IDENTIFICADOR INTERNO	NATURALEZA CONTABLE	CLAVES Y FAMILIAS UNSPSC INCLUIDAS
01	Software, Nube e Infraestructura TI	<code>software_ti</code>	Gasto Operativo	Servicios en la nube (AWS, GCP, Azure), licencias de software, hosting, dominios y desarrollo de sistemas (Segmento 81, Familias 8111, 4323).
02	Equipo de Cómputo y Electrónica	<code>computo_hardware</code>	Inversión / Activo Fijo	Computadoras, servidores, monitores, componentes electrónicos y periféricos (Segmentos 32, 43, Familias 4321, 4322).
03	Servicios Profesionales y Asesoría	<code>servicios_profesionales</code>	Gasto Operativo	Honorarios contables, asesoría legal, consultoría de gestión y servicios profesionales independientes (Segmento 80, Familias 8010, 8012, 8411).

04	Renta de Vehículos y Transporte	renta_vehiculos	Gasto Operativo	Arrendamiento de vehículos automotores utilitarios para actividades de trabajo (Clave 78111811).
05	Plataformas de Movilidad y Taxis	movilidad_taxis	Viáticos / Transporte	Servicios de transporte terrestre por aplicaciones digitales y taxis concesionados (Claves 78111804, 78111808).
06	Combustibles y Lubricantes	combustibles	Gasto Operativo	Gasolinas, diésel y lubricantes automotrices (Segmento 15, Familia 1510).
07	Seguros y Coberturas	seguros_polizas	Gasto Operativo	Pólizas de seguros empresariales, coberturas de responsabilidad civil y fianzas (Familia 8413, Clave 80161505).
08	Viáticos, Viajes y Peajes	viaticos_viajes	Viáticos	Peajes de autopistas (95111602), pasajes aéreos (78111502), hospedaje temporal (90111500) y consumos en viajes (90101500).
09	Otros Gastos Operativos	otros_operativos	Gasto General	Insumos de oficina, papelería, paquetería, servicios generales y gastos no clasificados en los rubros anteriores.

2. Algoritmo de Resolución Jerárquica en 4 Niveles

El módulo de clasificación `backend/app/catalogos/sat_catalogo.py` evalúa cada concepto de factura mediante una cascada determinista de cuatro niveles de precisión:



2.1 Niveles de Resolución:

- 01 Nivel 1 (Clave Específica):** Evaluación exacta del código UNSPSC de 8 dígitos contra la tabla de excepciones y reglas directas.
- 02 Nivel 2 (Familia UNSPSC):** Evaluación de los primeros 4 dígitos del código para agrupar partidas dentro de familias funcionales (ej. `8111` para servicios de software).
- 03 Nivel 3 (Segmento UNSPSC):** Evaluación de los primeros 2 dígitos del código para clasificación por sector industrial (ej. `15` para combustibles).
- 04 Nivel 4 (Análisis Semántico):** Evaluación de cadenas de texto en el campo `descripcion` mediante patrones de expresiones regulares precompilados, permitiendo clasificar conceptos con claves genéricas (`01010101`).

3. Sembrado y Persistencia del Catálogo SAT

El catálogo de 52,547 claves oficiales del SAT se almacena de forma relacional en la tabla `catalogo_sat_keys` de la base de datos:


```
CREATE TABLE catalog_sat_keys (  
    id VARCHAR PRIMARY KEY,  
    clave VARCHAR(8) UNIQUE NOT NULL,  
    descripcion VARCHAR NOT NULL,  
    categoria_id VARCHAR,  
    tipo VARCHAR  
);  
  
CREATE INDEX ix_catalog_sat_keys_clave ON catalog_sat_keys (clave);
```

Comandos de Poblado

```
# Inicialización y sembrado de la base de datos completa  
make db-empty  
  
# O mediante la interfaz de línea de comandos de Python  
PYTHONPATH=backend python3 -m app.cli seed-sat
```

tribuTACOS — 05. Especificación de API REST FastAPI

Contratos de servicios REST, parámetros de consulta, esquemas de respuesta JSON y códigos de estado HTTP.

1. Mapeo de Enrutadores y Servicios

La API está estructurada bajo el prefijo `/api` y expuesta por defecto en el puerto `8010`:



2. Especificación de Endpoints Principales

2.1 Resumen Fiscal Consolidado

GET `/api/summary`

Calcula el estado fiscal integral del ejercicio solicitado, consolidando ingresos de nómina, actividad profesional, gastos deducibles, deducciones personales y simulaciones anuales y mensuales.

- **Parámetros de Consulta:**
 - `year` (*string, requerido*): Ejercicio fiscal (ej. `2024`).
 - `client_id` (*string, opcional*): Identificador del cliente (por defecto: `default`).
- **Respuesta Exitosa (`200 OK`):**

```
{
  "client": {
    "id": "default",
    "name": "Sheila Shellaquiles Ortega",
    "rfc": "SHLL250825XYZ",
    "email": "tributacos@shellaquiles.org"
  },
  "year": "2024",
  "anios_disponibles": ["2021", "2022", "2023", "2024", "2025", "2026"],
  "sections": {
    "sueldos": {
      "total_ingresos": 402000.00,
      "gravado": 373150.00,
      "exento": 28850.00,
      "isr retenido": 55128.84,
      "detalle": []
    },
    "honorarios": {
      "ingresos": 186200.00,
      "isr retenido": 18620.00,
      "iva trasladado": 29792.00,
      "detalle": []
    },
    "reporte_gastos": [
      {
        "uuid": "4A1B2C3D-E4F5-6789-ABCD-EF0123456789",
        "emisor": "PROVEEDOR EJEMPLO SA DE CV",
        "subtotal": 1500.00,
        "iva": 240.00,
        "total": 1740.00,
        "categoria_gasto": {
          "id": "software_ti",
          "nombre": "Software, Nube e Infraestructura TI"
        }
      }
    ],
    "deducciones_personales": {
      "total": 71100.00,
      "deducible_efectivo": 71100.00,
      "tope_legal": 198031.80
    }
  },
  "simulacion_provisional_mensual": [
    {
      "mes_numero": 1,
      "mes_nombre": "Enero",
      "ingresos_periodo": 12300.00,
      "deducciones_bancarizadas_periodo": 5120.00,
      "base_gravable_provisional": 7180.00,
      "isr_a_cargo_mes": 0.00,
      "iva_a_cargo_mes": 0.00,
      "total_a_pagar_mes": 0.00
    }
  ],
  "simulacion_anual": {
    "ingresos_acumulables_totales": 494157.83,
    "deducciones_personales_efectivas": 71100.00,
    "base_gravable": 423057.83,
    "isr_tarifa": 71390.73,
    "retenciones_totales": 73748.84,
  }
}
```

```

    "saldo_a_favor": 2358.11,
    "saldo_a_cargo": 0.00
  }
}

```

2.2 Auditoría y Documentos Oficiales SAT

GET /api/sat_docs/summary

Retiene las declaraciones anuales oficiales, matriz de 12 pagos provisionales y conciliaciones bancarias registradas para el ejercicio.

- **Parámetros de Consulta:**

- **year** (*string, requerido*): Ejercicio fiscal (ej. **2024**).
- **client_id** (*string, opcional*): Identificador del cliente.

- **Respuesta Exitosa (**200 OK**):**

```

{
  "ejercicio": "2024",
  "declaracion_anual": {
    "num_operacion": "240500190419",
    "fecha_presentacion": "25/04/2025 17:30",
    "tipo_declaracion": "Normal",
    "ingresos_acumulables": 494157.83,
    "deducciones_personales": 71100.00,
    "base_gravable": 423057.83,
    "isr_tarifa": 71390.73,
    "saldo_a_favor": 2358.11,
    "saldo_a_cargo": 0.00,
    "clabe": "012180000000000000",
    "banco": "INSTITUCION BANCARIA MULTIPLE S.A."
  },
  "pagos_provisionales": [
    {
      "mes_numero": 1,
      "mes_nombre": "Enero",
      "num_operacion": "2401123456",
      "isr_a_cargo": 0.00,
      "iva_a_cargo": 0.00,
      "total_pagado": 0.00,
      "tiene_acuse": false
    }
  ]
}

```

2.3 Gestión de Contribuyentes

GET /api/clients

Lista los clientes o contribuyentes registrados en el sistema.

- **Respuesta Exitosa (**200 OK**):**

```
[
  {
    "id": "default",
    "name": "Sheila Shellaquiles Ortega",
    "rfc": "SHLL250825XYZ",
    "email": "tributacos@shellaquiles.org",
    "created_at": "2026-08-25T12:00:00"
  }
]
```

3. Códigos de Estado HTTP

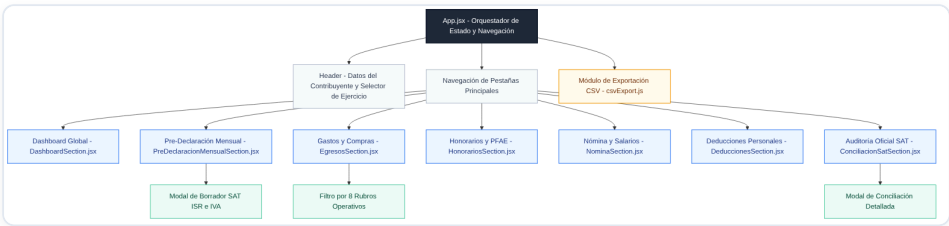
CÓDIGO	SIGNIFICADO	DESCRIPCIÓN
200 OK	Petición Exitosa	La solicitud se procesó correctamente y devuelve los datos requeridos.
400 Bad Request	Parámetros Inválidos	El ejercicio fiscal o los parámetros enviados no cumplen con el formato esperado.
404 Not Found	Recurso No Encontrado	No se encontraron datos para el ejercicio o cliente especificado.
500 Internal Error	Error de Servidor	Error no controlado durante el procesamiento contable o acceso a la base de datos.

tribuTACOS — 06. Frontend, UI/UX y Componentes Next.js

Arquitectura cliente, árbol de componentes, sistema de diseño con Tailwind CSS, paletas semánticas autodescriptivas, gestión de estado y exportación de datos.

1. Arquitectura de Componentes en Next.js

La capa de presentación está implementada en **Next.js 15 (App Router)** con **React 19**, estructurada mediante componentes desacoplados y modulares:



2. Sistema de Diseño y Paleta Semántica Autodescriptiva

El diseño visual está implementado con **Tailwind CSS**, estructurado mediante clases utilitarias y variables semánticas en `globals.css` e `index.css`. Se asocia una identidad cromática de alto contraste y formalidad a cada régimen fiscal y estado tributario:

CONCEPTO FISCAL	TONALIDAD	CLASES TAILWIND PRINCIPALES	SIGNIFICADO CONTABLE
Sueldos y Salarios	Azul cobalto	<code>bg-blue-50</code> , <code>border-blue-500</code> , <code>text-blue-700</code>	Ingresos estables por relación laboral subordinada.
Honorarios PFAE	Índigo / Violeta	<code>bg-indigo-50</code> , <code>border-indigo-500</code> , <code>text-indigo-700</code>	Servicios profesionales y actividad empresarial.
Gastos Deducibles	Esmeralda / Verde	<code>bg-emerald-50</code> , <code>border-emerald-500</code> , <code>text-emerald-700</code>	Egresos operativos que reducen la base gravable.
Gastos No Deducibles	Gris pizarra	<code>bg-slate-50</code> , <code>border-slate-300</code> , <code>text-slate-500</code>	Egresos sin requisitos fiscales o de uso personal.
Deducciones Personales	Ámbar / Dorado	<code>bg-amber-50</code> , <code>border-amber-500</code> , <code>text-amber-700</code>	Gastos médicos, lentes, SGMM y aportaciones al retiro (PPR).
Saldo a Favor (Devolución)	Verde bosque	<code>bg-green-100</code> , <code>border-green-600</code> , <code>text-green-800</code>	Devolución oficial determinada a favor del contribuyente.

Saldo a Cargo (Pago)	Rojo coral	<code>bg-rose-50</code> , <code>border-rose-500</code> , <code>text-rose-700</code>	Impuesto pendiente de liquidar ante el SAT.
----------------------	------------	---	---

3. Matriz de Pre-Declaración Mensual (`PreDeclaracionMensualSection.jsx`)

Este componente presenta la tabla analítica de los 12 meses del ejercicio con estados visuales claros:

- **Mes con Pérdida Operativa o Sin Pago:**
 - Fondo tenue (`bg-rose-50`).
 - Indicador de balance negativo `Déficit: -$X,XXX.XX` .
 - Estatus de resultado: `Sin Pago / Saldo a Favor` .
- **Mes con Superávit o Utilidad:**
 - Fondo blanco con borde esmeralda (`border-emerald-500`).
 - Indicador de utilidad `+$XX,XXX.XX` .
 - Monto a pagar al SAT: `Pagar: $X,XXX.XX` .

4. Motor de Exportación de Datos (`csvExport.js`)

El módulo `frontend/src/csvExport.js` genera archivos CSV estructurados con el **Byte Order Mark (BOM) UTF-8 (`\uFEFF`)**, garantizando compatibilidad inmediata con **Microsoft Excel**, **Apple Numbers** y **Google Sheets** sin problemas de codificación de caracteres en español.

Tipos de Reportes Disponibles:

- 01 Reporte Maestro de Egresos:** UUID, Fecha, RFC Emisor, Razón Social, Clave SAT, Descripción de Concepto, Rubro Operativo, Subtotal, IVA, Total y Estado de Deducibilidad.
- 02 Matriz de Pagos Provisionales (12 Meses):** Ingresos PFAE, Gastos Deducibles, Utilidad o Pérdida, ISR Retenido, ISR Causado, ISR a Pagar, IVA Cobrado, IVA Acreditable e IVA a Pagar.
- 03 Auditoría de Sueldos y Nómina:** Empleador, Quincenas pagadas, Sueldo Bruto, Percepciones Exentas, ISR Retenido y Cuotas IMSS.
- 04 Resumen de Deducciones Personales:** Categoría, Prestador de servicio, RFC, Importe y aplicabilidad al límite legal del Art. 151 LISR.